BASES DE DATOS II

Stored Procedures Avanzados

Handlers - SIGNAL - SQLSTATE - Transacciones

Prof. Lic. Adolfo Alonso

Full stackperiodista - Full Legacy Fanfan - Full Kleenex Corrier

UTN San Juan Frías
Ciclo Lectivo 2026

Trabajo Práctico Integrador
Ejercicios conceptuales y prácticos sobre manejo de errores y transacciones

Trabajo Práctico: Stored Procedures Avanzados

Objetivo: Analizar el comportamiento de handlers en MySQL, aprender a hacer los ejercicios conceptuales y aplicar los conceptos en la escritura de procedimientos almacenados con manejo de errores, transacciones, SIGNAL y validaciones de negocio (ejercicios prácticos).

1. Mecánica Fundamentales

- El parámetro entre errores que controla en el que se detiene.
- Lo handler cuál utiliza el bloque donde está definido.
- Lo SIGNAL sin handler termina el procedimiento.
- ROLLBACK no revierte operaciones.
- Error con handler atrapado sale el procedimiento.
- Error sin handler termina todo.
- REFERENCIAS: Internet.com.mx/mysqlref/ref/signal.
- Lo SIGNAL siempre termina el procedimiento a menos que lo atrapes.
- Cada procedimiento maneja sus propios errores.

2. Guía para analizar ejercicios conceptuales

- Identifica qué pasa cuando ocurre el error.
- Identifica si hay handler que atrapa el error.
- Evalúa si se ejecuta el COMMIT/ROLLBACK.
- Determina si el error se propaga.
- Determina qué llega después.
- Identifica qué código siguiente ejecutaría.

ANTES EJERCICIOS CONCEPTUALES 1-10:

Ejercicio 1:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_1()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error capturado';
    END;

    SELECT 'Inicio';
    INSERT INTO tabla VALUES (1);
    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_1();
```

¿Qué se imprime?

¿Se corre la ejecución? ¿Porque?

Ejercicio 2:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_2()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error capturado';
    END;

    SELECT 'Inicio';
    INSERT INTO tabla VALUES (1);
    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_2();
```

¿Qué se imprime?

¿Se corre la ejecución? ¿Porque?

Ejercicio 3:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_3()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error capturado';
    END;

    SELECT 'Inicio';
    INSERT INTO tabla VALUES (1);
    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_3();
```

¿Qué se imprime?

¿Se corre la ejecución? ¿Porque?

Ejercicio 4:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_4()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error capturado';
    END;

    SELECT 'Inicio';
    INSERT INTO tabla VALUES (1);
    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_4();
```

¿Qué se imprime?

¿Se corre la ejecución? ¿Porque?

Ejercicio 5:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_5()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error capturado';
    END;

    SELECT 'Inicio';
    INSERT INTO tabla VALUES (1);
    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_5();
```

¿Qué se imprime si tiene `p_error=1`?

¿Se corre la ejecución? ¿Porque?

Ejercicio 6:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_6()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error capturado';
    END;

    SELECT 'Inicio';
    INSERT INTO tabla VALUES (1);
    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_6();
```

¿Qué se imprime?

¿Se corre la ejecución? ¿Porque?

Ejercicio 7:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_7()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error capturado';
    END;

    SELECT 'Inicio';
    INSERT INTO tabla VALUES (1);
    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_7();
```

¿Qué se imprime?

¿Se corre la ejecución? ¿Porque?

Ejercicio 8:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_8()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error capturado';
    END;

    SELECT 'Inicio';
    INSERT INTO tabla VALUES (1);
    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_8();
```

¿Qué se imprime?

¿Se corre la ejecución? ¿Porque?

Ejercicio 9:

```
DELIMITER $$
CREATE PROCEDURE ejemplo_9()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error';
    END;

    SELECT 'Inicio';

    CALL ejemplo_9();

    SELECT 'Fin';
END$$
DELIMITER ;

CALL ejemplo_9();
```

¿Qué se imprime si tiene `p_error=1`?

¿Se corre la ejecución? ¿Porque?

Ejercicio 10 (DIVIDIRPOR2 y ROLLBACK parcial)

```
DELIMITER $$
CREATE PROCEDURE ejemplo_10(
    IN p_numero INT,
    OUT p_resultado VARCHAR(100))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT CONCAT('Error capturado. Número: ', p_numero);
        ROLLBACK;
        SET p_resultado = CONCAT('Error en número: ', p_numero);
    END;

    SELECT CONCAT('Procesando número: ', p_numero);
    INSERT INTO tabla VALUES (p_numero);
    SELECT CONCAT('Insertado número: ', p_numero);
    UPDATE tabla SET valor = valor / 2 WHERE id = p_numero;
    SELECT CONCAT('Dividido número: ', p_numero);
    COMMIT;
    SET p_resultado = 'Todo OK';
END$$
DELIMITER ;

CALL ejemplo_10(1, @resultado);
SELECT @resultado;
```

¿Qué `COMMIT` **se ejecuta?** `p_numero` **operaciones se flag** `id` **en la tabla de** `id` **requiere que** `id` **sea** `id` **que** `id` **que** `id` **que